

Árboles Binarios

Los árboles binarios son el caso particular más simple. Son usados para representar ciertos tipos de expresiones algebraicas, algoritmos, búsquedas y ordenamientos.

Definición: Un conjunto T de elementos (nodos) es un árbol binario si:

1. T es vacío, o
2. T está particionado en 3 conjuntos disjuntos
 - a) un solo elemento R , llamado la raíz
 - b) 2 conjuntos que son árboles binarios, llamados los subárboles izquierdo y derecho de R

de esta forma

T es un árbol binario si

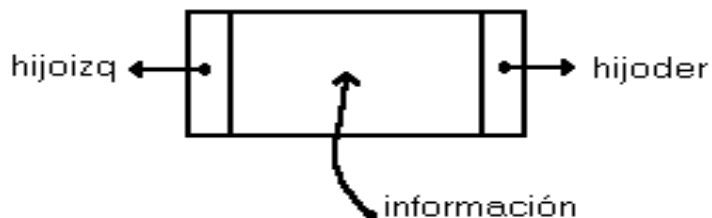
1. T no tiene nodos, o
2. T es de la forma:



Donde n es un nodo y T_I y T_D son árboles binarios.

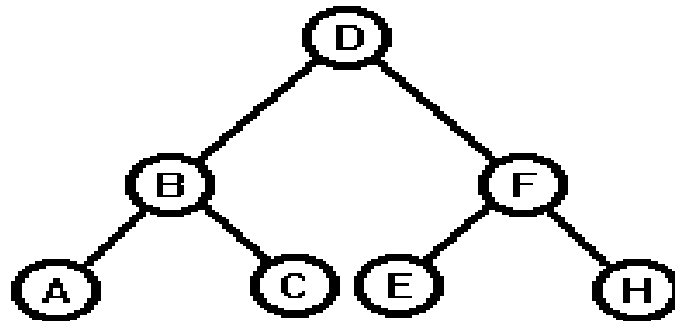
5.2.1 Representación ligada de árboles binarios

La representación más usada es la ligada o dinámica, donde cada nodo o celda tiene la siguiente forma:

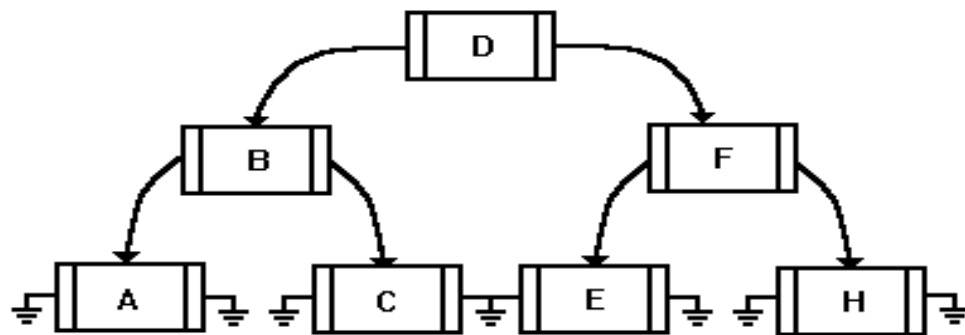


Un nodo de un árbol puede representarse como:

Ejemplo:



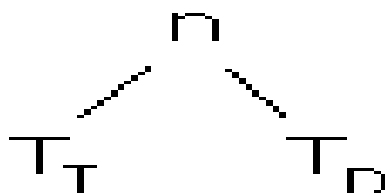
El árbol anterior, en representación ligada:



Árboles Binarios de Búsqueda

Definición: Dado un conjunto de nodos (con un campo asignado como llave de búsqueda) T es un árbol binario de búsqueda si:

1. T es vacío, o
2. T es de la forma



En donde:

- a) la llave de búsqueda de n es mayor que todas las llaves de búsqueda de T_I
- b) la llave de búsqueda de n es menor que todas las llaves de búsqueda de T_D
- c) T_I y T_D son árboles binarios de búsqueda

Recorrido :

Una vez construido un árbol binario de búsqueda surge la necesidad de recorrerlo, es decir una manera sistemática de visitar cada nodo del árbol. La forma en la cual una lista lineal se recorre es trivial (del primero al último, o viceversa). Sin embargo, para recorrer un árbol, esta forma natural no puede aplicarse.

Para recorrer un árbol, existen varias formas de lograrlo, las 3 más comunes son **preorden**, **inorden** y **postorden**. Estos métodos difieren en el orden en el cual los nodos son visitados. Siguiendo la costumbre de empezar a visitar antes lo que se encuentra a la izquierda que lo de la derecha, se tienen 3 posibilidades de visitar a la raíz (antes, en medio o después).

Definición: Para recorrer un árbol binario no vacío T en **preorden o previo**:

1. visitar la raíz
2. recorrer recursivamente el subárbol izquierdo
3. recorrer recursivamente el subárbol derecho

Definición: Para recorrer un árbol binario no vacío T en **inorden o simétrico**:

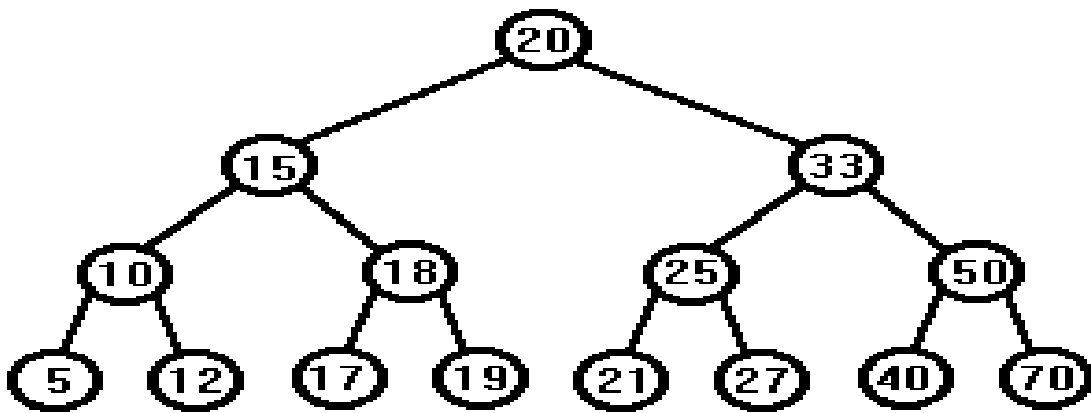
1. recorrer recursivamente el subárbol izquierdo
2. visitar la raíz
3. recorrer recursivamente el subárbol derecho

Definición: Para recorrer un árbol binario no vacío T en **postorden o posterior**:

1. recorrer recursivamente el subárbol izquierdo
2. recorrer recursivamente el subárbol derecho
3. visitar la raíz

Ejemplo:

Considere el siguiente árbol:



el resultado que se obtiene al recorrer el árbol en

preorden:

20, 15 10, 5, 12, 18, 17, 19, 33, 25, 21, 27, 50, 40, 70

inorden:

5, 10, 12, 15, 17, 18, 19, 20, 21, 25, 27, 33, 40, 50, 70

postorden:

5, 12, 10, 17, 19, 18, 15, 21, 27, 25, 40, 70, 50, 33, 20

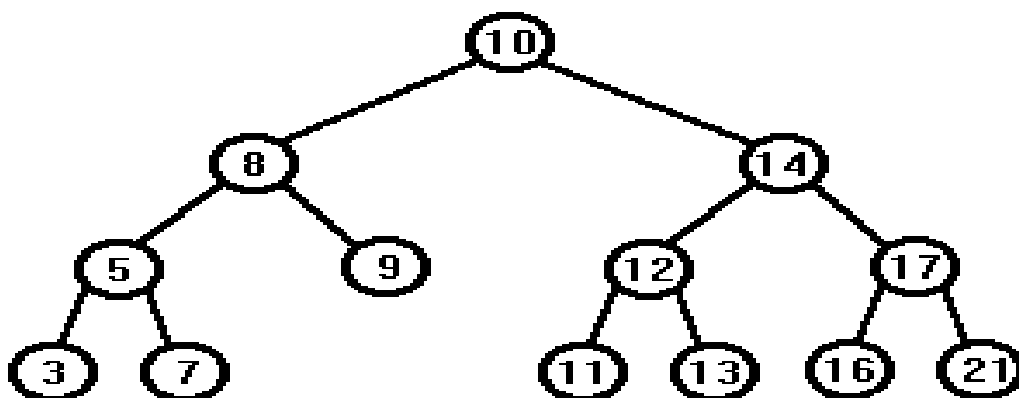
Buscar:

Una ventaja de los árboles binarios de búsqueda es que el algoritmo de búsqueda surge de una manera natural. Si x es la llave de la raíz de un árbol binario de búsqueda y se está buscando a x , se ha terminado la búsqueda; esto es cierto suponiendo que las llaves de búsqueda son únicas. Si se busca una llave menor que x , entonces debe encontrarse en el subárbol izquierdo. Similarmente, si se busca una llave mayor que x , debe buscarse en el subárbol derecho.

Así, puede procederse recursivamente hasta encontrar la llave buscada. El siguiente algoritmo de búsqueda supone que no existen llaves duplicadas y regresa una referencia al nodo donde se encuentra la llave de búsqueda o el valor de nil si la llave no se encuentra en el árbol.

Ejemplo:

Considere el siguiente árbol binario de búsqueda:



Si se busca la llave 7 en el árbol se compara 7 con 10 y se desciende por la izquierda; se compara 7 con 8 y se desciende por la izquierda; al comparar 7 con 5 se desciende por la derecha y se encuentra la llave.

Insertar:

La inserción en un árbol binario de búsqueda es similar a la búsqueda. Suponga que se desea insertar una nueva llave con valor x , lo primero que debe hacerse es una búsqueda, si x se encuentra en el árbol no se requiere insertarla de nuevo (si se desea que las llaves del árbol sean únicas); en otro caso la búsqueda termina no exitosamente en una hoja o en un nodo con un solo subárbol; y así un nuevo nodo que contenga la llave x debe ser insertado debajo de esa hoja, como hijo derecho o hijo izquierdo, dependiendo del valor de x .

Siguiendo el esquema anterior, se podría modificar el algoritmo de búsqueda anterior, para que regrese también la hoja donde termina la búsqueda, es decir la raíz de donde se debe insertar el nuevo nodo.

Borrar:

Por lo general, borrar siempre resulta más complicado. Lo primero que debe hacerse para borrar un nodo de un árbol binario de búsqueda es buscarlo. Si la búsqueda no fue exitosa, no hay nada que hacer; sin embargo, si existe un nodo x con la llave de búsqueda que se desea borrar, con padre P , entonces ese nodo puede:

- a) ser una hoja
- b) tener un único hijo
- c) tener dos hijos

A continuación se presenta cada uno de éstos casos para su estudio.

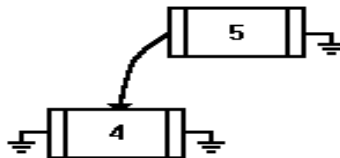
- a) El nodo a eliminar es una hoja

Este caso es el más sencillo, en el cual lo único que hay que hacer, es que la liga que lo referencia debería ser nil.

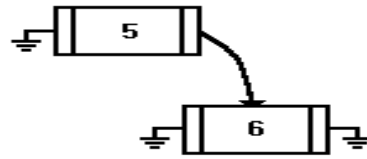
Considere el siguiente árbol:



Si se desea borrar el nodo con llave de búsqueda 6 el árbol queda como:

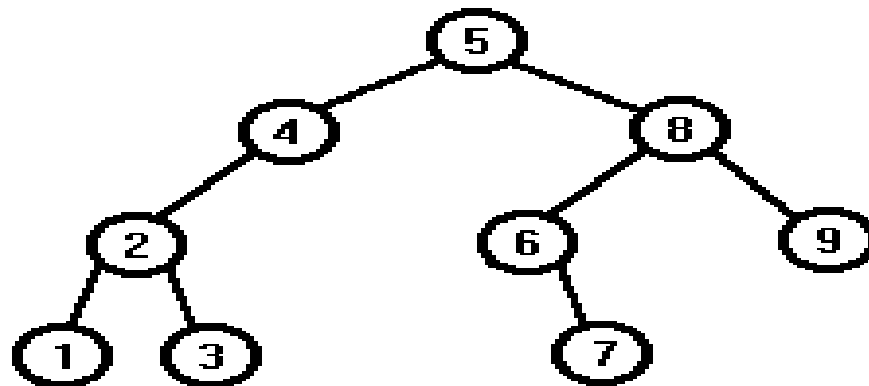


Si se hubiera eliminado el nodo con llave de búsqueda 4 del árbol original entonces el árbol debería quedar como:

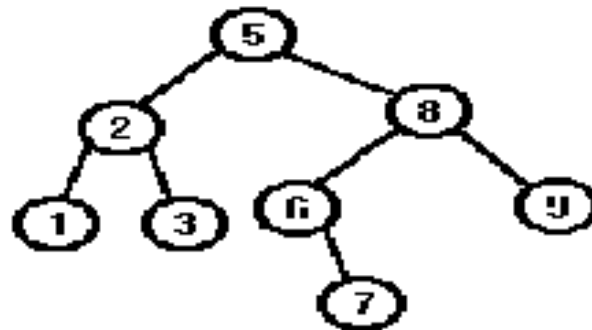


b) El nodo que se desea eliminar tiene un único hijo

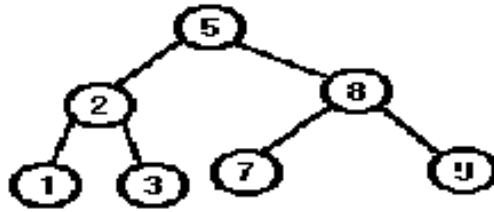
En éste caso para eliminar un nodo su padre deberá referenciar a su hijo. Por ejemplo considere el siguiente árbol:



Si se desea eliminar el nodo con llave de búsqueda 4, se debe cambiar el subárbol izquierdo de 5 por el subárbol que contiene a 2 como raíz.



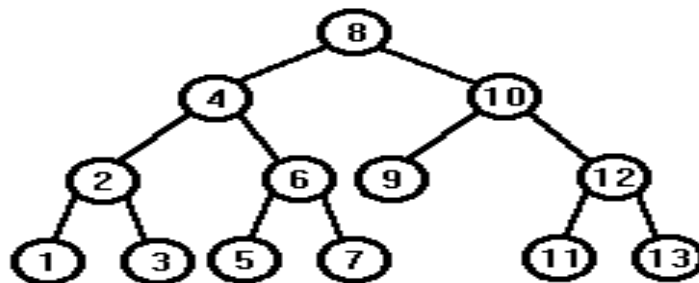
Esto mismo ocurre cuando se trata de eliminar un nodo con un único subárbol izquierdo, suponga ahora que deseamos eliminar el nodo cuya información es 6 del árbol mostrado en la figura anterior. En este caso bastaría con que el subárbol izquierdo de 8 referenciará al subárbol derecho del 6 (nodo 7). Hecho lo anterior el nuevo árbol se presenta a continuación:



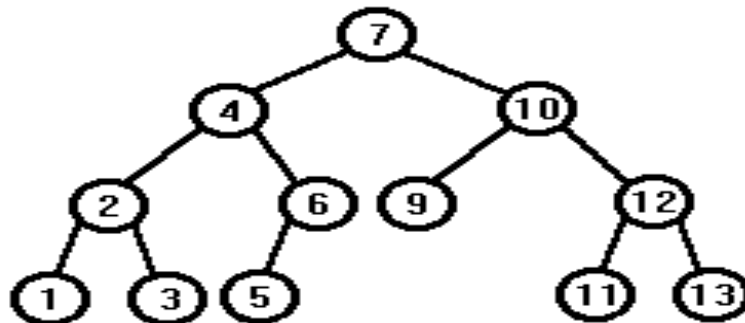
c) El nodo que se desea eliminar tiene dos hijos

En este caso se debe sustituir el valor de la información del nodo que se desea borrar por el nodo que contiene la menor llave que se encuentra en el subárbol derecho o bien la llave mayor del subárbol izquierdo, esto es necesario para no perder la propiedad de árbol binario de búsqueda. En cualquier elección que se tome, (el menor de los mayores o el mayor de los menores), el nodo que contiene esa información a lo más tendrá un hijo, por lo que su eliminación es fácil de realizar.

Considere el siguiente árbol:



Si se desea eliminar la raíz (que tiene 2 hijos) del árbol anterior, se puede substituir la llave 8 por la 7, y después eliminar el nodo que lo contiene, es decir, el árbol quedaría como:



O bien sustituirla por el menor de los mayores, es decir, la llave 9, el árbol quedaría:

